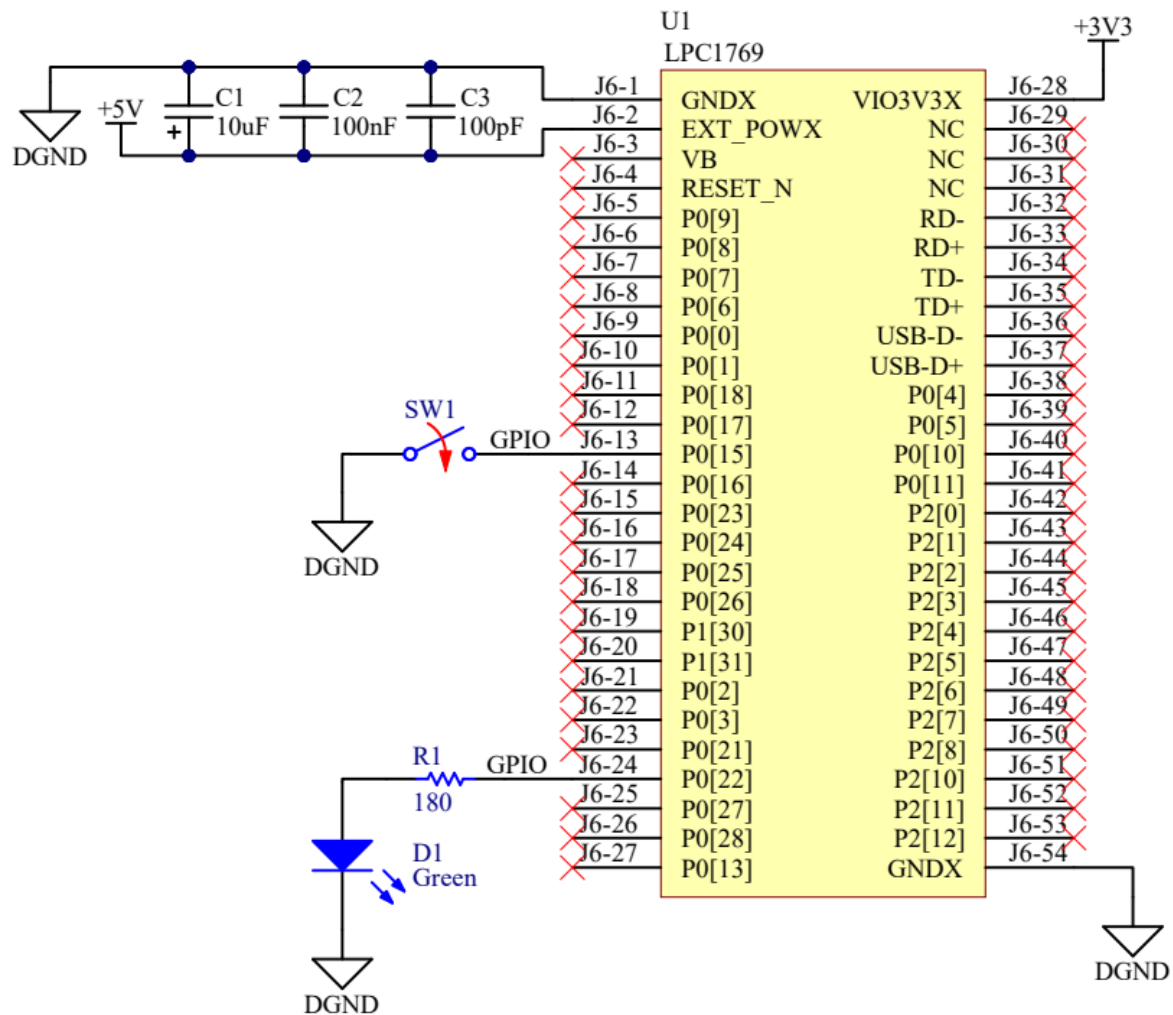


```

1  /*=====
2  * @file    Ejercicio_3_1.c
3  *
4  * @brief    Controla un LED conectado al pin P0.22, alternando periódicamente
5  *           entre los estados encendido y apagado.
6  *
7  * @details  - El tiempo de intermitencia es determinado por el estado lógico
8  *           de una entrada digital conectada al pin P0.15, vía interrupciones
9  *           por GPIO.
10 *           - Implementar retardo bloqueante.
11 *
12 * @author   Ing. E. Migliore
13 *
14 * @version  3.0
15  */=====

```



```

1  /*=====
2  * DIRECTIVAS DE INCLUSIÓN
3  *=====*/
4  #include "LPC17xx.h"
5
6  /*=====
7  * DEFINICIÓN DE CONSTANTES Y MACROS
8  *=====*/
9  #define NVIC_PRIO_BITS      5U  /* Bits físicos del registro de prioridad:
10                                     2^5 = 32 niveles de prioridad posibles
11                                     (del 0 al 31)*/
12  #define EINT3_PRIORITY      1U  //Prioridad 1 para Interrupción Externa EINT3
13  #define EINT3_IRQ           21  //Número de IRQ (Interrupt Request) para EINT3
14
15  #define DELAY_FAST          1000000
16  #define DELAY_SLOW          4000000
17
18  /*=====
19  * DEFINICIÓN DE TIPO DE DATOS Y VARIABLES GLOBALES
20  *=====*/
21  volatile uint32_t delayTime = DELAY_SLOW;
22
23  /*=====
24  * PROTOTIPOS DE FUNCIONES
25  *=====*/
26  void cfgGPIO(void);
27  void cfgIntGPIO(void);
28  static inline void cfgNVIC(int32_t, uint32_t);
29  void blinkLed(uint32_t);
30  void delay(uint32_t);
31
32  /*=====
33  * FUNCIÓN PRINCIPAL
34  *=====*/
35  int main(void)
36  {
37      /*-----
38      * INICIALIZACIÓN DEL MCU
39      *-----*/
40      cfgGPIO();
41      cfgIntGPIO();
42
43      /*-----
44      * INICIO PROGRAMA PRINCIPAL
45      *-----*/
46      while(1)
47      {
48          blinkLed(delayTime);
49      }
50
51      return 0;
52  }
53
54  /*=====
55  * DEFINICIONES DE FUNCIONES
56  *=====*/
57
58  /**
59   * @brief   Configura los pines P0.15 como entrada digital y P0.22 como
60   *          salida digital.
61   */
62  void cfgGPIO(void)
63  {
64      LPC_PINCON->PINSEL0 &= ~(0b11<<30); //P0.15 GPIO Function
65      LPC_PINCON->PINSEL1 &= ~(0b11<<12); //P0.22 GPIO Function
66      LPC_GPIO0->FIODIR  &= ~(1<<15);    //P0.15 Digital Input
67      LPC_GPIO0->FIODIR  |= (1<<22);     //P0.22 Digital Output
68
69      return;

```

```

70 }
71
72 /*****
73  * @brief   Configura el NVIC y SCB para la habilitación de interrupciones.
74  *****/
75 static inline void cfgNVIC(int32_t irq_number, uint32_t priority_number)
76 {
77     //Asigna Prioridad de Interrupción
78     if(irq_number < 0) {
79         //Ejecuta interrupciones del sistema.
80         //SHP = System Handler Priority Registers
81         SCB->SHP[((uint32_t)(irq_number) & 0xF) - 4] = ((irq_number << (8 -
            NVIC_PRIO_BITS)) & 0xFF);
82     } else {
83         //Ejecuta interrupciones de periféricos.
84         //IP = Interrupt Priority Registers
85         NVIC->IP[(uint32_t)(irq_number)] = ((priority_number << (8 - NVIC_PRIO_BITS)) &
            0xFF);
86     }
87
88     //Habilitación de Interrupción
89     NVIC->ISER[((uint32_t)irq_number >> 5)] = (1 << ((uint32_t)irq_number & 0x1F));
90 }
91
92 /*****
93  * @brief   Configura la interrupción por GPIO en el pin P0.15.
94  *
95  * @details - Habilita la detección de interrupción por flanco descendente en el
96  *             pin P0.15.
97  *             - Limpia banderas pendientes previas, asigna la prioridad
98  *             en el NVIC y habilita el vector EINT3_IRQn.
99  *****/
100 void cfgIntGPIO(void)
101 {
102     LPC_GPIOINT->IO0IntEnF |= (1<<15); //INTGPIO P0.15 por Flanco Descendente
103     LPC_GPIOINT->IO0IntClr |= (1<<15); //Limpia Flag INTGPIO P0.15
104
105     cfgNVIC(EINT3_IRQ, EINT3_PRIORITY);
106
107     return;
108 }
109
110 /*****
111  * @brief   Rutina de Servicio de Interrupción (ISR) para el vector EINT3.
112  *
113  * @details - Atiende las interrupciones por GPIO del Puerto 0. Verifica si la
114  *             interrupción proviene específicamente del pin P0.15 por flanco ascendente.
115  *             - Alterna el tiempo de retardo de un LED mediante una variable `index`
116  *             y limpia la bandera de interrupción en el periférico.
117  *****/
118 void EINT3_IRQHandler(void)
119 {
120     static uint32_t index = 0;
121
122     //Consulta Puerto de Interrupción
123     if(LPC_GPIOINT->IntStatus & (1<<0)){ //Se produjo una interrupción en P0?
124         //Consulta Pin de Interrupción
125         if(LPC_GPIOINT->IO0IntStatF & (1<<15)){ //Se produjo una interrupción en P0.15?
126             //Selección del tiempo de retardo según el estado de index
127             index++;
128             if(index%2){
129                 delayTime = DELAY_FAST;
130             }
131             else{
132                 delayTime = DELAY_SLOW;
133             }
134
135             LPC_GPIOINT->IO0IntClr |= (1<<15); //Limpia Flag INTGPIO P0.15
136         }
137     }

```

```

137     }
138     return;
139 }
140
141 /*****
142  * @brief Ejecuta un ciclo de intermitencia del LED conectado a P0.22.
143  *****/
144 void blinkLed(uint32_t delayTime)
145 {
146     LPC_GPIO0->FIOSET |= (1<<22); // P0.22: LED ON
147     delay(delayTime);
148     LPC_GPIO0->FIOCLR |= (1<<22); // P0.22: LED OFF
149     delay(delayTime);
150
151     return;
152 }
153
154 /*****
155  * @brief Genera un retardo bloqueante.
156  *
157  * @details Ejecuta un lazo de espera durante la cantidad de iteraciones
158  * indicada por delayTime. La duración del retardo depende del
159  * valor recibido y de la frecuencia de CPU.
160  *****/
161 void delay(uint32_t delayTime)
162 {
163     for(volatile uint32_t i = 0; i < delayTime; i++)
164     {
165         __NOP(); // NOP: 1 ciclo de CPU
166     }
167
168     return;
169 }
170
171

```